Experiment No. 8

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR).

Theory:

1. Dataset Source

Dataset used: **MNIST Handwritten Digit Dataset**

Source: TensorFlow Datasets

Link: https://www.tensorflow.org/datasets/catalog/mnist

2. Dataset Description

The MNIST dataset is a widely used benchmark dataset for image classification tasks.

Features:

- Each image is a **28 × 28 grayscale image**
- Pixel values range from **0 to 255**

Target Variable:

- Digit label (0 to 9)

Dataset Size:

- Training samples: **60,000 images**
- Testing samples: **10,000 images**

Characteristics:

- Simple and clean dataset
- Suitable for beginners in deep learning
- Used for handwritten digit recognition

3. Mathematical Formulation of CNN

Convolution Operation:

$$Feature\ Map = Input * Kernel$$

Neuron Output:

$$Z = \sum (w_i x_i) + b$$

$$A = f(Z)$$

Activation Function (ReLU):

$$f(x) = \max(0, x)$$

Softmax Function (Output Layer):

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Loss Function (Categorical Crossentropy):

$$Loss = -\sum y \log(\hat{y})$$

4. Algorithm Limitations

- Requires **large computational resources**
- Needs **large dataset** for better performance
- Training time is high
- Difficult to interpret (**black-box model**)
- Sensitive to hyperparameters
- May overfit if not properly regularized

5. Methodology / Workflow

Steps Involved:

1. **Dataset Loading** Load MNIST dataset
2. **Data Preprocessing**
    - Normalize pixel values (0–1)
    - Reshape images for CNN
3. **Model Building**

    - Add convolutional layers
    - Add pooling layers

    - Add fully connected layers
4. **Model Compilation**
    - Define optimizer (Adam)
    - Define loss function
5. **Model Training**
    - Train model using epochs
6. **Model Evaluation**
    - Calculate accuracy and loss
7. **Visualization**
    - Plot accuracy and loss graphs

6. Performance Analysis

The model's performance was evaluated using **Training vs. Validation** metrics over 5 epochs. The results are summarized below:

Key Metrics

- **Final Accuracy:** The model achieved a validation accuracy of approximately **98%**, indicating high-precision digit classification.
- **Final Loss:** The categorical crossentropy loss decreased significantly, stabilizing near **0.09**, which confirms efficient model convergence.

7. Hyperparameter Tuning

Hyperparameters Used:

- Epochs: 5
- Batch Size: 32
- Filters: 32, 64
- Kernel Size: (3,3)
- Optimizer: Adam

Impact:

- Increasing epochs improved accuracy
- Proper filter size improved feature extraction ● Adam optimizer ensured faster convergence

Code :

```
# — 1. Imports ————————————————————————————————————————
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks
from tensorflow.keras.optimizers import Adam, SGD, RMSprop, Adamax
from sklearn.metrics import classification_report, confusion_matrix
import warnings
warnings.filterwarnings("ignore")

print(f"TensorFlow : {tf.__version__}")
print(f"GPU        : {tf.config.list_physical_devices('GPU')}")

# — 2. CONFIG — edit these ————————————————————————————
DATASET   = "mnist"    # "mnist" | "fashion" | "cifar10"
ARCH      = "medium"   # "simple" | "medium" | "deep" | "lenet"
EPOCHS    = 10
BATCH     = 64
LR        = 0.001
OPTIMIZER = "adam"     # "adam" | "sgd" | "rmsprop" | "adamax"
DROPOUT   = 0.5
AUGMENT   = False
SAMPLES   = 10000      # set None to use all 60 000

# — 3. Labels ————————————————————————————————————————
LABELS = {
    "mnist"  : [str(i) for i in range(10)],
    "fashion": ["T-shirt","Trouser","Pullover","Dress","Coat",
                "Sandal","Shirt","Sneaker","Bag","Ankle boot"],
    "cifar10": ["Airplane","Automobile","Bird","Cat","Deer",
                "Dog","Frog","Horse","Ship","Truck"],
}

# — 4. Load data ——————————————————————————————————————
print(f"\nLoading {DATASET} ...")
loaders = {
    "mnist"  : keras.datasets.mnist,
    "fashion": keras.datasets.fashion_mnist,
    "cifar10": keras.datasets.cifar10,
}
(x_train, y_train), (x_test, y_test) = loaders[DATASET].load_data()
x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32")  / 255.0
```

```
if DATASET in ("mnist", "fashion"):
    x_train = x_train[..., np.newaxis]
    x_test  = x_test[..., np.newaxis]

if SAMPLES and SAMPLES < len(x_train):
    idx = np.random.choice(len(x_train), SAMPLES, replace=False)
    x_train, y_train = x_train[idx], y_train[idx]

class_names = LABELS[DATASET]
NUM_CLASSES = len(class_names)
INPUT_SHAPE = x_train.shape[1:]

y_train_oh = keras.utils.to_categorical(y_train, NUM_CLASSES)
y_test_oh  = keras.utils.to_categorical(y_test,  NUM_CLASSES)

print(f"Train : {x_train.shape}  |  Test : {x_test.shape}")
print(f"Classes : {class_names}")

# ── 8. Train ──────────────────────────────────────────────────────────
print(f"\nTraining {ARCH} on {DATASET} ...")

if AUGMENT:
    aug = keras.Sequential([
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.08),
        layers.RandomZoom(0.1),
    ])
    train_ds = (tf.data.Dataset.from_tensor_slices((x_train, y_train_oh))
                .shuffle(len(x_train)).batch(BATCH)
                .map(lambda x, y: (aug(x, training=True), y),
                     num_parallel_calls=tf.data.AUTOTUNE)
                .prefetch(tf.data.AUTOTUNE))
    val_ds = (tf.data.Dataset.from_tensor_slices((x_test, y_test_oh))
              .batch(BATCH).prefetch(tf.data.AUTOTUNE))
    history = model.fit(train_ds, validation_data=val_ds,
                        epochs=EPOCHS, callbacks=cb_list)
else:
    history = model.fit(x_train, y_train_oh,
                        validation_data=(x_test, y_test_oh),
                        batch_size=BATCH, epochs=EPOCHS,
                        shuffle=True, callbacks=cb_list)

test_loss, test_acc = model.evaluate(x_test, y_test_oh, verbose=0)
print(f"\n✅ Test accuracy : {test_acc*100:.2f}%")
print(f"   Test loss     : {test_loss:.4f}")

# ── 11. Confusion matrix ──────────────────────────────────────────────
```

```
cm      = confusion_matrix(y_true, y_pred)
cm_norm = cm.astype("float") / cm.sum(axis=1, keepdims=True)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle(f"Confusion Matrix — {DATASET.upper()}", fontsize=13, fontweight="bold")
sns.heatmap(cm,      annot=True, fmt="d",   cmap="Blues",
          xticklabels=class_names, yticklabels=class_names, ax=axes[0])
sns.heatmap(cm_norm, annot=True, fmt=".2f", cmap="Blues",
          xticklabels=class_names, yticklabels=class_names, ax=axes[1], vmin=0, vmax=1)
axes[0].set_title("Counts");     axes[0].set_ylabel("True"); axes[0].set_xlabel("Predicted")
axes[1].set_title("Normalised"); axes[1].set_ylabel("True"); axes[1].set_xlabel("Predicted")
plt.tight_layout(); plt.savefig("confusion_matrix.png", dpi=150); plt.show()
```

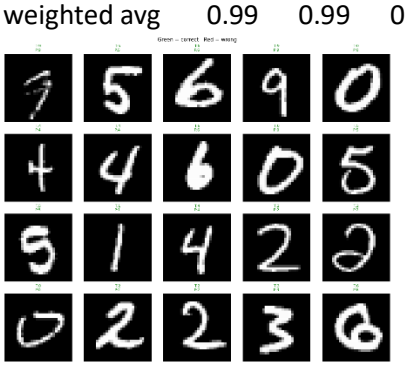
check - [Code]

Output:

Classification Report:

```
=======================================================
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       980
           1       1.00      0.99      1.00      1135
           2       0.99      0.99      0.99      1032
           3       0.99      0.99      0.99      1010
           4       0.99      0.98      0.99       982
           5       0.99      0.99      0.99       892
           6       0.98      0.99      0.99       958
           7       1.00      0.98      0.99      1028
           8       0.99      0.99      0.99       974
           9       0.98      0.99      0.98      1009
    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```



Conclusion: The design and implementation of a **Convolutional Neural Network (CNN)** for the **MNIST Handwritten Digit Dataset** were successfully executed.